

01. 基础知识

算法设计与分析

课程概述

- ▶ 算法：解决特定问题的一系列清晰、有限的指令序列
- ▶ 算法五大特性：有穷性、确定性、可行性、输入、输出
- ▶ 算法分析：评估正确性、时间效率、空间效率、鲁棒性
- ▶ 核心目标：在有限时间和空间资源内获得正确结果

时间复杂度

- ▶ 衡量运行时间随输入规模 n 的增长趋势
- ▶ 关注点： $n \rightarrow \infty$ 时的渐近行为，而非精确毫秒数
- ▶ 基本操作：赋值、比较、算术运算视为 $O(1)$
- ▶ 分析步骤：找出最深层循环 $\rightarrow$ 确定迭代次数 $\rightarrow$ 忽略低阶项和常数

空间复杂度

- ▶ 衡量算法执行所需的额外内存空间（不含输入本身）
- ▶ 原地算法 (In-place): $O(1)$ 额外空间，如冒泡排序
- ▶ 辅助空间：递归调用栈、临时数组、哈希表等
- ▶ 空间换时间：用额外存储减少计算，典型如动态规划

渐近记号 — O 大O

- ▶ 定义: $f(n) = O(g(n))$, 存在 $c > 0, n_0$, 使得 $\forall n \geq n_0, f(n) \leq c \cdot g(n)$
- ▶ 含义: 上界, 最坏情况保证, 算法不会超过这个增长速度
- ▶ 示例: $3n^2 + 2n + 5 = O(n^2)$, 取 $c=4, n_0=3$ 即可满足
- ▶ 注意: O 描述的是上界, 不一定是最紧的上界

渐近记号 — Ω 与 Θ

- ▶ Ω (大Omega): 下界, $f(n) \geq c \cdot g(n)$, 最好情况保证
- ▶ Θ (大Theta): 紧确界, 同时是上下界, $f(n) = \Theta(g(n))$
- ▶ 关系: $f(n) = \Theta(g(n)) \iff f(n) = O(g(n))$ 且 $f(n) = \Omega(g(n))$
- ▶ o (小o): 严格上界, $f(n)/g(n) \rightarrow 0$, 如 $n = o(n^2)$

常见复杂度等级

- ▶ $O(1)$ — 常数：哈希表查找、数组索引访问
- ▶ $O(\log n)$ — 对数：二分查找、平衡二叉树操作
- ▶ $O(n)$ — 线性：遍历数组、链表、桶排序
- ▶ $O(n \log n)$ — 线性对数：归并排序、堆排序、快速排序平均
- ▶ $O(n^2)$ — 平方：冒泡排序、选择排序、Floyd部分情况
- ▶ $O(n^3)$ — 立方：矩阵乘法朴素算法、Floyd全源最短路径
- ▶ $O(2^n)$ — 指数：子集枚举、哈密顿路径
- ▶ $O(n!)$ — 阶乘：全排列枚举、旅行商问题暴力

复杂度分析实例

【单层循环】 for i=1 to n: $O(n)$

【嵌套循环】 for i=1 to n: for j=1 to n: $O(n^2)$

【对数循环】 while n>1: n=n/2: $O(\log n)$

【混合结构】 for i=1 to n: while j<n: $j*=2 \rightarrow O(n \log n)$

递归与迭代对比

- ▶ 递归：函数调用自身，代码结构清晰，接近数学定义
- ▶ 递归代价：系统栈开销，深度过大导致栈溢出
- ▶ 迭代：循环结构，显式维护状态，空间可控
- ▶ 尾递归：可被编译器优化为迭代，避免栈增长
- ▶ 选择：递归适合树/图遍历、分治；迭代适合线性处理、大数据量

数学基础 — 级数求和

- ▶ 等差数列: $1 + 2 + \dots + n = n(n+1)/2 = \Theta(n^2)$
- ▶ 等比数列: $1 + 2 + 4 + \dots + 2^n = 2^{n+1} - 1 = \Theta(2^n)$
- ▶ 调和级数: $1 + 1/2 + 1/3 + \dots + 1/n \approx \ln(n) + \gamma = \Theta(\log n)$
- ▶ 对数级数: $\log 1 + \log 2 + \dots + \log n = \log(n!) = \Theta(n \log n)$

数学基础 — 递推关系

- ▶ 展开法：逐层展开递推式，寻找规律，求和得解
- ▶ 代入法：猜测解的形式 $T(n) \leq cn \cdot \log n$ ，用归纳法验证
- ▶ 递归树法：画出递归调用树，计算每层代价，各层求和
- ▶ 变量替换：令 $m = \log n$ ，将复杂递推转化为熟悉形式

算法正确性证明

- ▶ 循环不变式：初始化保持、迭代保持、终止时给出正确结果
- ▶ 数学归纳法：基例成立 + 归纳假设 + 归纳步骤
- ▶ 反证法：假设算法不正确，推出矛盾
- ▶ 示例：插入排序的循环不变式——子数组 $A[1..j-1]$ 始终有序